

Verifying Wait-Freedom for Concurrent Higher-Order Programs

Egor Namakonov, Lars Birkedal, Amin Timany



AARHUS
UNIVERSITY

ECOOP'26
July 1, 2026

Wait-freedom: the strongest progress guarantee

Wait-freedom: the strongest progress guarantee

“An [implementation of] operation [on a concurrent data structure] is wait-free if it guarantees that every call finishes its execution in a finite number of steps.”

Herlihy and Shavit (2012)

Wait-freedom: the strongest progress guarantee

“An [implementation of] operation [on a concurrent data structure] is wait-free if it guarantees that every call finishes its execution in a finite number of steps.”

Herlihy and Shavit (2012)

(* Does the left thread always terminate? *)

`cnt.incr () || while true do cnt.incr (); done`

Wait-freedom: the strongest progress guarantee

“An [implementation of] operation [on a concurrent data structure] is wait-free if it guarantees that every call finishes its execution in a finite number of steps.”

Herlihy and Shavit (2012)

(* Does the left thread always terminate? *)

`cnt.incr () || while true do cnt.incr (); done`

`let new_cnt_blocking () =`

`let c = ref 0 in`

`let l = new_lock () in`

`let i _ =`

`acquire l;`

`let n = !c in`

`c := n + 1;`

`release l;`

`n`

`in { incr = i }`

X

Wait-freedom: the strongest progress guarantee

“An [implementation of] operation [on a concurrent data structure] is wait-free if it guarantees that every call finishes its execution in a finite number of steps.”

Herlihy and Shavit (2012)

(* Does the left thread always terminate? *)

`cnt.incr () || while true do cnt.incr (); done`

<code>let new_cnt_blocking () =</code>	<code>let new_cnt_lockfree () =</code>
<code> let c = ref 0 in</code>	<code> let c = ref 0 in</code>
<code> let l = new_lock () in</code>	<code> let rec i _ =</code>
<code> let i _ =</code>	<code> let n = !c in</code>
<code> acquire l;</code>	<code> if CAS(c, n, n + 1)</code>
<code> let n = !c in</code>	<code> then n</code>
<code> c := n + 1;</code>	<code> else i ()</code>
<code> release l;</code>	<code> in { incr = i }</code>
<code> n</code>	
<code> in { incr = i }</code>	

X

X

Wait-freedom: the strongest progress guarantee

"An [implementation of] operation [on a concurrent data structure] is wait-free if it guarantees that every call finishes its execution in a finite number of steps."

Herlihy and Shavit (2012)

(* Does the left thread always terminate? *)

`cnt.incr () || while true do cnt.incr (); done`

<code>let new_cnt_blocking () = let c = ref 0 in let l = new_lock () in let i _ = acquire l; let n = !c in c := n + 1; release l; n in { incr = i }</code>	<code>let new_cnt_lockfree () = let c = ref 0 in let rec i _ = let n = !c in if CAS(c, n, n + 1) then n else i () in { incr = i }</code>	<code>let new_cnt_waitfree () = let c = ref 0 in let i _ = FAA(c, 1) in { incr = i }</code>
--	--	---



Wait-free implementations are the hardest

Wait-free implementations are the hardest

Blocking queue,
Vindum and
Birkedal (2021)

```
let cg_enqueue lock head x =  
  let rec enqueue_go x head =  
    match head with  
    | None  
    -> Some (x, None)  
    | Some p -> Some (fst p, enqueue_go x (snd p))  
  in  
    acquire lock;  
    head := enqueue_go x !head;  
    release lock  
  
let cg_dequeue lock head () =  
  acquire lock;  
  let v = match !head with  
  | None  
  -> None  
  | Some p -> head := snd p; Some (fst p)  
  in  
    release lock;  
    v  
  
let cg_queue () =  
  let head = ref None in  
  let lock = newlock () in  
  (cg_dequeue lock head, cg_enqueue lock head x)
```

Wait-free implementations are the hardest

Blocking queue, Vindum and Birkedal (2021)

```
let cg_enqueue lock head x =
  let rec enqueue_go x head =
    match head with
    | None
    -> Some (x, None)
    | Some p -> Some (fst p, enqueue_go x (snd p))
  in
  acquire lock;
  head := enqueue_go x !head;
  release lock

let cg_dequeue lock head () =
  acquire lock;
  let v = match !head with
  | None
  -> None
  | Some p -> head := snd p; Some (fst p)
  in
  release lock;
  v

let cg_queue () =
  let head = ref None in
  let lock = newlock () in
  (cg_dequeue lock head, cg_enqueue lock head x)
```

Lock-free Michael-Scott queue, version by Vindum and Birkedal (2021)

```
let get_value x =
  match x with
  | None
  -> let rec spin () = spin () in spin ()
  | Some v -> v

let ms_dequeue toSent toTail =
  let rec loop () =
    let head = !toSent in
    let tail = !toTail in
    let toNext = snd (get_value !head) in
    let p = newProp in
    let next = !toNext in
    if head = (Resolve (!toSent) p ()) then
      if head = tail then
        match !next with
        | None -> None
        | Some _ -> CAS toTail tail next; loop ()
      end
    else
      if CAS toSent head next then
        Some (get_value (fst (get_value !next)))
      else loop ()
    else loop ()
  in loop ()

let ms_enqueue toTail x =
  let node = ref (Some (Some x, ref (ref None))) in
  let rec loop () =
    let tail = !toTail in
    let toNext = snd (get_value !tail) in
    let next = !toNext in
    if tail = !toTail then
      match !next with
      | None ->
        if CAS toNext next node then
          (CAS toTail tail node; {})
        else loop ()
      | Some _ -> CAS toTail tail next; loop ()
    end
  else loop ()
  in loop ()

let ms_queue () =
  let node
  = ref (Some (None, ref (ref None))) in
  let toTail = ref node in
  let toSent = ref node in
  ((ms_dequeue toSent toTail), (ms_enqueue toTail))
```

Wait-free implementations are the hardest

Blocking queue,
Vindum and
Birkedal (2021)

```

let cg_enqueue lock head x =
  let rec enqueue_go x head =
    match head with
    | None
    | Some p -> Some (fst p, enqueue_go x (snd p))
  in
  acquire lock;
  head := enqueue_go x !head;
  release lock

let cg_dequeue lock head () =
  acquire lock;
  let v = match !head with
  | None
  -> None
  | Some p -> head := snd p; Some (fst p)
  in
  release lock;
  v

let cg_queue () =
  let lock = ref None in
  let lock = new_lock () in
  (cg_dequeue lock head, cg_enqueue lock head x)

```

Lock-free Michael-Scott queue, version by Vindum and Birkedal (2021)

```

let get_value x =
  match x with
  | None
  => let rec spin () = spin () in spin ()
  | Some v -> v

let ms_dequeue toSnt toTail =
  let rec loop () =
    let head = !toSnt in
    let tail = !toTail in
    let toNext = snd (get_value !head) in
    let p = NewProp in
    let next = !toNext in
    if head = (Resolve (!toSnt) p ()) then
      if head = tail then
        match !next with
        | None -> None
        | Some _ -> CAS toTail tail next; loop ()
      end
    else
      if CAS toSnt head next then
        Some (get_value (fst (get_value !next)))
      else loop ()
    else loop ()
  in loop ()

let ms_enqueue toTail x =
  let node = ref (Some (Some x, ref (ref None))) in
  let rec loop () =
    let tail = !toTail in
    let toNext = snd (get_value !tail) in
    let next = !toNext in
    if tail = !toTail then
      match !next with
      | None ->
        CAS toNext next node then
          (CAS toTail tail node; ())
        else loop ()
      | Some _ -> CAS toTail tail next; loop ()
    end
  else loop ()
  in loop ()

let ms_queue () =
  let node
  = ref (Some (None, ref (ref None))) in
  let total = ref node in
  let toSnt = ref node in
  ((ms_dequeue toSnt total). (ms_enqueue toTail

```

Wait-free queue, Kogan and Petrank (2011)
(translated w/ Claude)

```

type node = { mutable value: int; mutable next: state option ref;
              mutable enqTid: int; mutable deqTid: int }
type op_desc = { phase: int; pending: bool; enqueue: bool; node: node option }

let head =
  ref (mk_sentinel ())
let tail =
  ref (mk_sentinel ())
let state = Array.init num_threds
  (fun i -> { phase = -1; pending = false; enqueue = true; node = None })

let max_phase () =
  Array.fold_left (fun m d -> max m d.phase) (-1) state

let is_still_pending tid ph =
  state.(tid).pending && state.(tid).phase <= ph

let help_phase =
  Array.iteri (fun i d ->
    if d.pending && d.phase <= phase then
      if d.enqueue then help_enq i phase
    else
      help_deq i phase
  ) state

let enq value =
  let phase = max_phase () + 1 in
  state.(tid) <- { phase; pending = true; enqueue = true;
                  node = Some { value; next = ref None;
                               enqTid = tid; deqTid = -1 } };

  help_phase;
  help_finish_enq ()

let help_enq tid phase =
  while is_still_pending tid phase do
    let last = !tail in
    let next = !(last.next) in
    if last = !tail then
      match next with
      | None ->
        if is_still_pending tid phase then
          if CAS last.next None (state.(tid).node) then
            help_finish_enq ()
      | Some _ ->
        help_finish_enq ()
    done

  let help_finish_enq () =
    let last = !tail in
    match !(last.next) with
    | None -> ()
    | Some next ->
      let tid = next.enqTid in
      let cur_desc = state.(tid) in
      if last = !tail && state.(tid).node = Some next then begin
        CAS state.(tid).cur_desc { cur_desc with pending = false;
                                     CAS tail last next
                                     end
        let deq () =
          let phase = max_phase () + 1 in
          state.(tid) <- { phase; pending = true; enqueue = false; node = None }
          help_phase;
          help_finish_deq ();
          match state.(tid).node with
          | None
            -> raise Empty
            | Some node -> node.next.value
        let help_deq tid phase =
          while is_still_pending tid phase do
            let first = !head in
            let last = !tail in
            let next =
              if (first.next) in
                if first = !head then
                  match next with
                  | None ->
                    let cur_desc = state.(tid) in
                    if last = !tail && is_still_pending tid phase then
                      CAS state.(tid).cur_desc
                        { cur_desc with pending = false; node = None }
                    | Some _ -> help_finish_enq ()
                  else
                    let cur_desc = state.(tid) in
                    let node =
                      if is_still_pending tid phase then begin
                        if first = !head && node <= Some first then
                          if not (CAS state.(tid).cur_desc
                              { cur_desc with node = Some first }) then ()
                        else begin
                          CAS first.deqTid (-1) tid;
                          help_finish_deq ()
                        end
                      end
                    done
                let help_finish_deq () =
                  let first = !head in
                  match !(first.next) with
                  | None -> ()
                  | Some next ->
                    let tid = first.deqTid in
                    if tid <= -1 then begin
                      let cur_desc = state.(tid) in
                      if first = !head then begin
                        CAS state.(tid).cur_desc { cur_desc with pending = false;
                                                         CAS head first next
                                                         end
                      end
                    end
                end
            end
          end
        end
      end
    end
  end

```

Wait-freedom is hard to even define

“An operation is wait-free if it guarantees that every call finishes its execution in a finite number of steps.” [Herlihy and Shavit (2012)]

Wait-freedom is hard to even define

Is the data structure initialized?

What if operation was stored in heap?

How to define formally in an ML-like language?

"An operation is wait-free if it guarantees that every call finishes its execution in a finite number of steps." [Herlihy and Shavit (2012)]

What about safety?

How the execution is scheduled?

Who calls it? Is it an arbitrary client program?

Wait-freedom is hard to even define

Is the data structure initialized?

What if operation was stored in heap?

How to define formally in an ML-like language?

“An operation is wait-free if it guarantees that every call finishes its execution in a finite number of steps.” [Herlihy and Shavit (2012)]

What about safety?

How the execution is scheduled?

Who calls it? Is it an arbitrary client program?

$\text{waitFree}(\text{op}) : \text{Prop}$

Wait-freedom is hard to even define

Is the data structure initialized?

What if operation was stored in heap?

How to define formally in an ML-like language?

"An operation is wait-free if it guarantees that every call finishes its execution in a finite number of steps." [Herlihy and Shavit (2012)]

What about safety?

How the execution is scheduled?

Who calls it? Is it an arbitrary client program?

$\text{waitFree}_c(\text{op}) : \text{Prop}$

need to account for initialization;
omit this parameter for simplicity

Wait-freedom is hard to even define, harder to prove.

 $\vdash \text{waitFree}(\text{op})$

Wait-freedom is hard to even define, harder to prove.
Solution: verify the operation with Hoare-style logic instead

$$\begin{array}{ccc} \vdash & \text{WaitFreeSpec}(\text{op}) & \Rightarrow \\ \text{⚖️} & & \text{🦉} \end{array} \quad \vdash \text{waitFree}(\text{op})$$

Wait-freedom is hard to even define, harder to prove.
Solution: verify the operation with Hoare-style logic instead

$$\text{⊢} \text{WaitFreeSpec}(\text{op}) \Rightarrow \text{⊢} \text{waitFree}(\text{op})$$

Our contributions, all mechanized in Rocq:

- Definition $\text{waitFree}(\text{op})$ of op 's wait-freedom in higher-order setting;
- Specification $\text{WaitFreeSpec}(\text{op})$ in Lawyer logic and the corresponding adequacy theorem;
- Case studies for simple algorithms;
- Adapting the approach for verifying realistic, *not exactly wait-free* algorithms

Recent work: proving termination with Lawyer logic

Recent work: proving termination with Lawyer logic

Layer logic for termination reasoning

Trillium framework for refinement reasoning

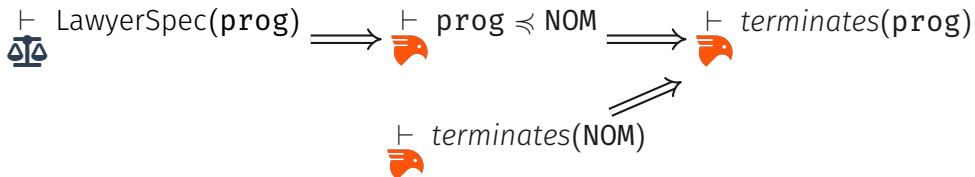
Iris framework for higher-order separation logics

} “Lawyer logic” in this talk

Recent work: proving termination with Lawyer logic

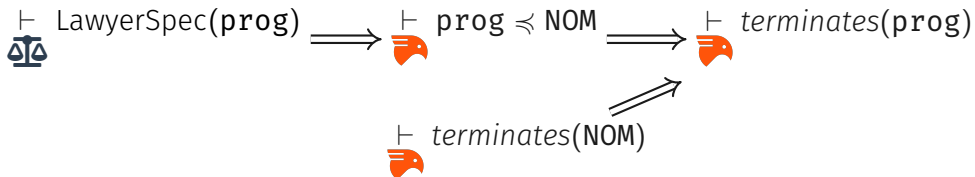
Recent work: proving termination with Lawyer logic

Lawyer approach, restricted to non-blocking concurrency:



Recent work: proving termination with Lawyer logic

Lawyer approach, restricted to non-blocking concurrency:



LTS	Language	NOM ("No-Obligations Model")
State	thread pool and heap	finite <i>fuel</i> assigned to each thread
Labels	thread ids	thread ids
Transitions	small-step semantics	crucially, <i>burning fuel on every step</i>

Challenge: we verify an open program in unknown environment

Challenge: we verify an open program in unknown environment

Key property for refining closed **prog** to NOM

$$\begin{array}{ccc} \vdash_{\text{⚖️}} \text{LawyerSpec}(\mathbf{prog}) & \implies & \vdash_{\text{⚖️}} \begin{array}{l} \text{execution of } \mathbf{prog} \text{ preserves invariants} \\ \text{and refinement to NOM} \end{array} \end{array}$$

Challenge: we verify an open program in unknown environment

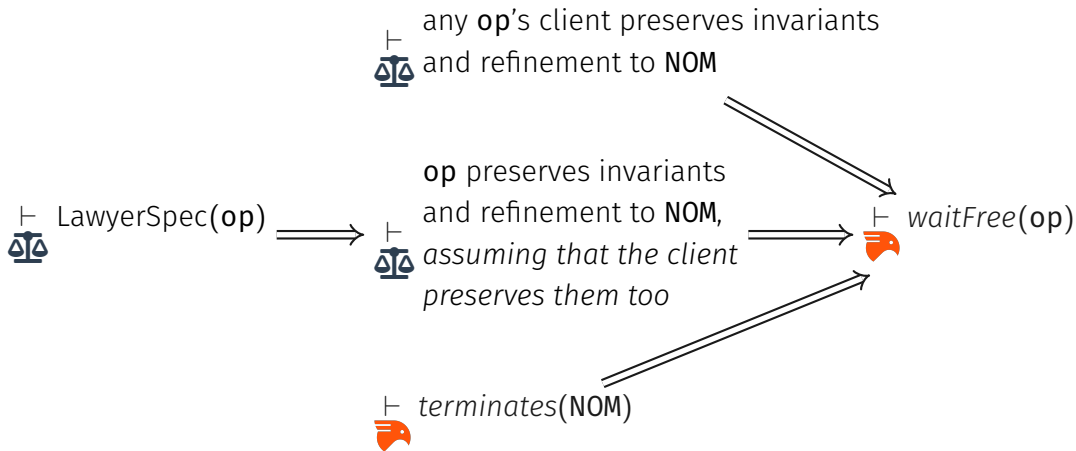
Key property for refining closed **prog** to NOM

$$\begin{array}{ccc} \vdash \text{LawyerSpec}(\mathbf{prog}) & \Longrightarrow & \vdash \begin{array}{l} \text{execution of } \mathbf{prog} \text{ preserves invariants} \\ \text{and refinement to NOM} \end{array} \end{array}$$

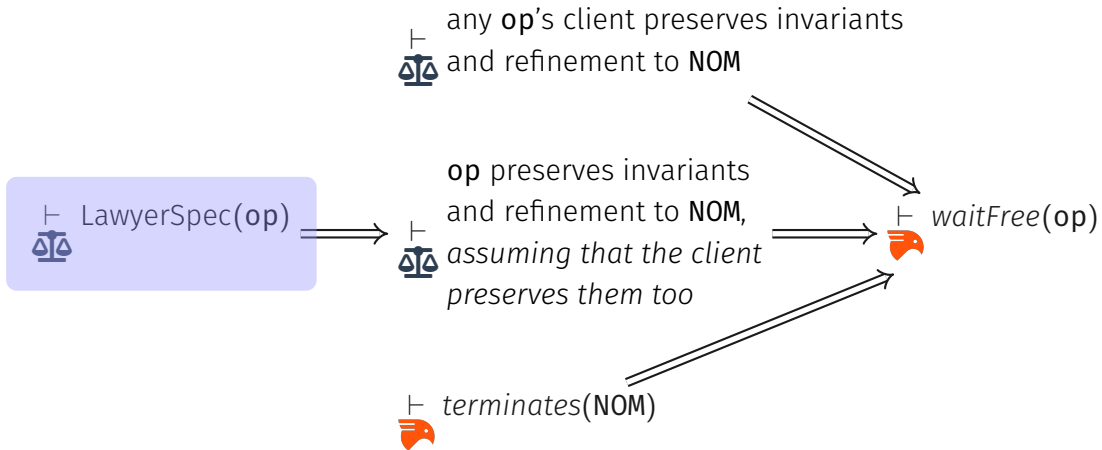

turns into a weaker property for standalone wait-free **op**:

$$\begin{array}{ccc} \vdash \text{LawyerSpec}(\mathbf{op}) & \Longrightarrow & \vdash \begin{array}{l} \text{execution of } \mathbf{op} \text{ preserves invariants} \\ \text{and refinement to NOM,} \\ \text{assuming that client of } \mathbf{op} \text{ preserves them too} \end{array} \end{array}$$


User proves spec and applies our new adequacy theorem



User proves spec and applies our new adequacy theorem



Reusing Lawyer logic to prove $\vdash \text{LawyerSpec}(\text{incr})$



let incr x =

FAA(c, 1)

Reusing Lawyer logic to prove $\vdash \text{LawyerSpec}(\text{incr})$



For every $\tau : \text{Thread}$ and $x : \text{Val}$:

$\{ \text{Mem}_{\tau} * \text{Mem}_{\tau} * \boxed{\exists n : \mathbb{Z}. c \mapsto n} \}$

let incr x =

FAA(c, 1)

$\{\text{True}\}$

Reusing Lawyer logic to prove $\vdash \text{LawyerSpec}(\text{incr})$



For every $\tau : \text{Thread}$ and $x : \text{Val}$:

$\{ \text{fuel}_{\tau} * \text{fuel}_{\tau} * \boxed{\exists n : \mathbb{Z}. c \mapsto n} \}$

let incr x =

FAA(c, 1)

$\{\text{True}\}$

Amount of fuel and
concrete invariant
depend on operation

“phase” resource
omitted for simplicity

Reusing Lawyer logic to prove $\vdash \text{LawyerSpec}(\text{incr})$



For every $\tau : \text{Thread}$ and $x : \text{Val}$:

$\{ \text{Mem}_{\tau} * \text{Mem}_{\tau} * \boxed{\exists n : \mathbb{Z}. c \mapsto n} \}$

let incr x =

FAA(c, 1)

$\{\text{True}\}$

Reusing Lawyer logic to prove $\vdash \text{LawyerSpec}(\text{incr})$



For every $\tau : \text{Thread}$ and $x : \text{Val}$:

$\{ \text{fuel}_\tau * \text{fuel}_\tau * \boxed{\exists n : \mathbb{Z}. c \mapsto n} \}$

let incr x =

$\{ \text{fuel}_\tau \}$

FAA(c, 1)

$\{\text{True}\}$

$\{\text{True}\}$

LAWYER-BETA-FUEL

$$\frac{\{P\} e[v/x] \{Q\}^\tau}{\{ \text{fuel}_\tau * P \} (\lambda x. e) v \{Q\}^\tau}$$

Reusing Lawyer logic to prove $\vdash \text{LawyerSpec}(\text{incr})$



For every $\tau : \text{Thread}$ and $x : \text{Val}$:

$\{\text{⊞}_{\tau} * \text{⊞}_{\tau} * \boxed{\exists n : \mathbb{Z}. c \mapsto n}^{\mathcal{N}}\}$

let incr x =

$\{\text{⊞}_{\tau}\}$

$\{\text{⊞}_{\tau} * \exists n. c \mapsto n\}$

FAA(c, 1)

$\{\exists n. c \mapsto n\}_{\top \setminus \mathcal{N}}$

$\{\text{True}\}_{\top}$

$\{\text{True}\}_{\top}$

LAWYER-BETA-FUEL

$$\frac{\{P\} e[v/x] \{Q\}_{\mathcal{E}}^{\tau}}{\{\text{⊞}_{\tau} * P\} (\lambda x. e) v \{Q\}_{\mathcal{E}}^{\tau}}$$

LAWYER-INV-ACC-TL

$$\frac{\{P * Q\} e \{P * R\}_{\mathcal{E} \setminus \mathcal{N}}^{\tau} \quad e \text{ is atomic} \quad \mathcal{N} \subseteq \mathcal{E} \quad \dots}{\{\boxed{P}^{\mathcal{N}} * Q\} e \{R\}_{\mathcal{E}}^{\tau}}$$

Reusing Lawyer logic to prove $\vdash \text{LawyerSpec}(\text{incr})$



For every $\tau : \text{Thread}$ and $x : \text{Val}$:

$$\{\text{⊖}_{\tau} * \text{⊖}_{\tau} * \boxed{\exists n : \mathbb{Z}. c \mapsto n}^{\mathcal{N}}\}$$

let incr x =

$$\{\text{⊖}_{\tau}\}$$

$$\{\text{⊖}_{\tau} * \exists n. c \mapsto n\}$$

FAA(c, 1)

$$\{\exists n. c \mapsto n\}_{\top \setminus \mathcal{N}}$$

$$\{\text{True}\}_{\top}$$

$$\{\text{True}\}_{\top}$$

LAWYER-BETA-FUEL

$$\frac{\{P\} e[v/x] \{Q\}_{\mathcal{E}}^{\tau}}{\{\text{⊖}_{\tau} * P\} (\lambda x. e) v \{Q\}_{\mathcal{E}}^{\tau}}$$

LAWYER-INV-ACC-TL

$$\frac{\{P * Q\} e \{P * R\}_{\mathcal{E} \setminus \mathcal{N}}^{\tau} \quad e \text{ is atomic} \quad \mathcal{N} \subseteq \mathcal{E} \quad \dots}{\{\boxed{P}^{\mathcal{N}} * Q\} e \{R\}_{\mathcal{E}}^{\tau}}$$

LAWYER-FAA-FUEL

$$\{\text{⊖}_{\tau} * \ell \mapsto n\} \text{FAA}(\ell, k) \{m. m = n * \ell \mapsto (n + k)\}_{\mathcal{E}}^{\tau}$$

Reusing Lawyer logic to prove $\vdash \text{LawyerSpec}(\text{incr})$



For every $\tau : \text{Thread}$ and $x : \text{Val}$:

$\{\text{⊞}_{\tau} * \text{⊞}_{\tau} * \boxed{\exists n : \mathbb{Z}. c \mapsto n}^{\mathcal{N}}\}$

let incr x =

$\{\text{⊞}_{\tau}\}$

$\{\text{⊞}_{\tau} * \exists n. c \mapsto n\}$

FAA(c, 1)

$\{\exists n. c \mapsto n\}_{\top \setminus \mathcal{N}}$

$\{\text{True}\}_{\top}$

$\{\text{True}\}_{\top}$



LAWYER-BETA-FUEL

$$\frac{\{P\} e[v/x] \{Q\}_{\mathcal{E}}^{\tau}}{\{\text{⊞}_{\tau} * P\} (\lambda x. e) v \{Q\}_{\mathcal{E}}^{\tau}}$$

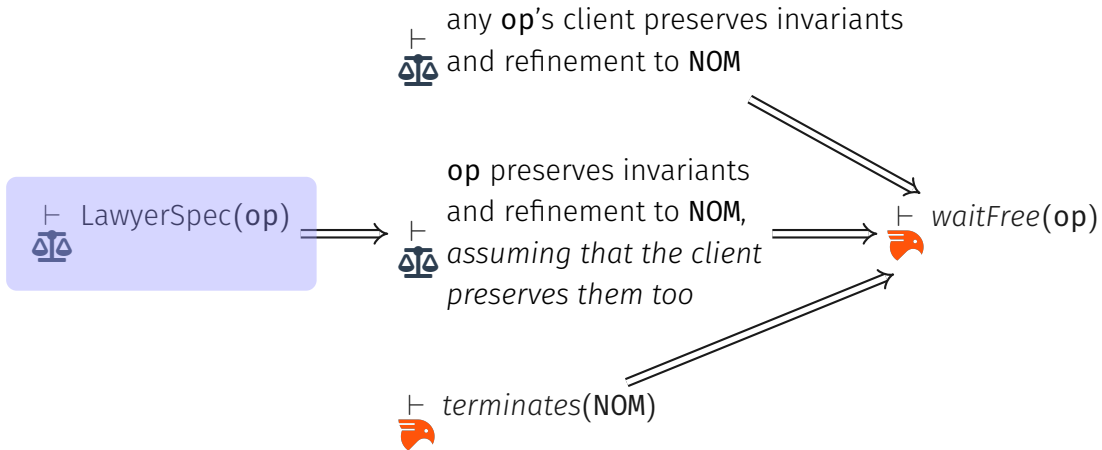
LAWYER-INV-ACC-TL

$$\frac{\{P * Q\} e \{P * R\}_{\mathcal{E} \setminus \mathcal{N}}^{\tau} \quad e \text{ is atomic} \quad \mathcal{N} \subseteq \mathcal{E} \quad \dots}{\{\boxed{P}^{\mathcal{N}} * Q\} e \{R\}_{\mathcal{E}}^{\tau}}$$

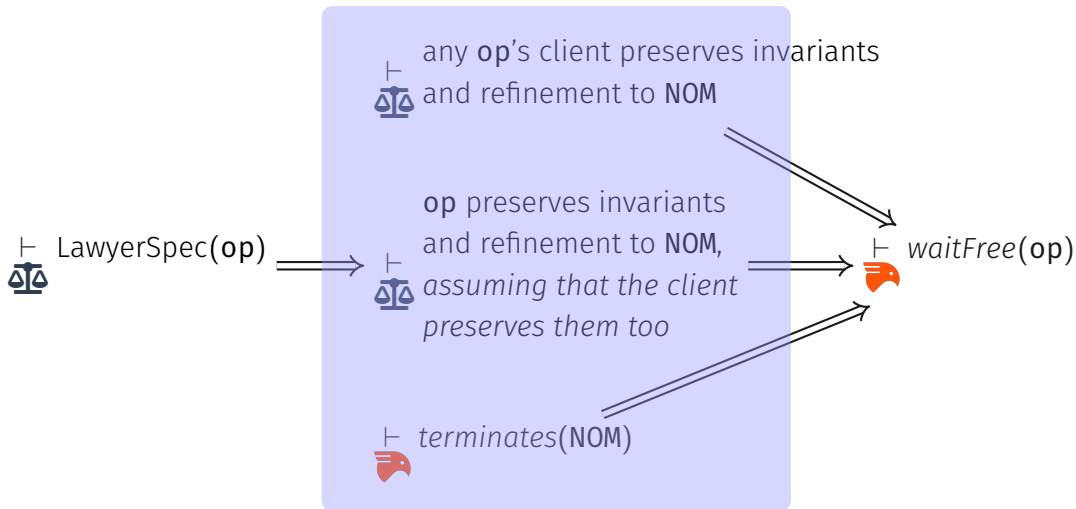
LAWYER-FAA-FUEL

$$\{\text{⊞}_{\tau} * \ell \mapsto n\} \text{FAA}(\ell, k) \{m. m = n * \ell \mapsto (n + k)\}_{\mathcal{E}}^{\tau}$$

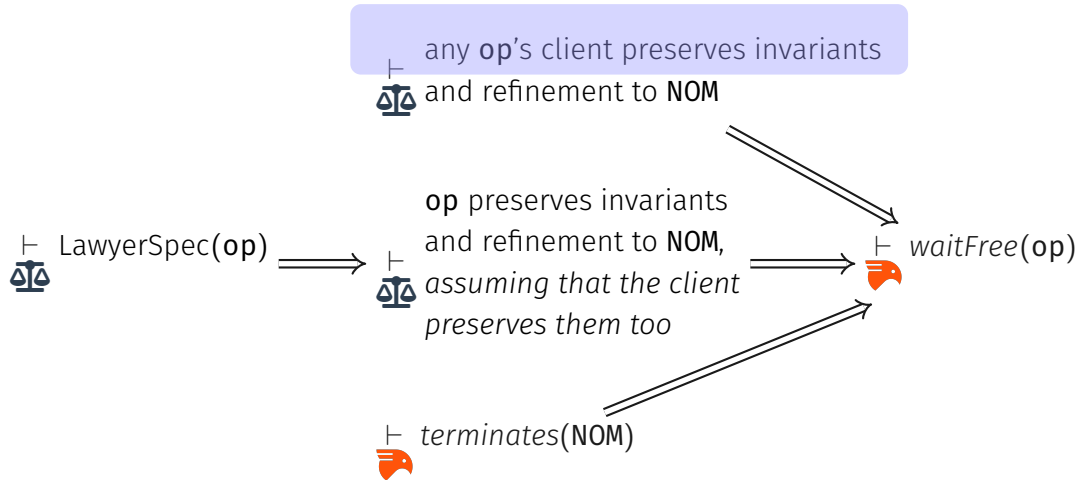
User proves spec and applies our new adequacy theorem



User proves spec and applies our new adequacy theorem



User proves spec and applies our new adequacy theorem



Invariants of the wait-free operation must be preserved

E.g. safe termination of **let** `incr x = FAA(c, 1)`
relies on $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ being preserved by its client

Invariants of the wait-free operation must be preserved,
i.e. the operation should be “robustly safe”

E.g. safe termination of **let** `incr x = FAA(c, 1)`
relies on $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ being preserved by its client

Robust safety of verified **op**, more formally:

 $\vdash \forall \text{prog} : \text{Expr}. \text{LawyerSpec}(\text{op}) \multimap \text{PresInv}(\text{prog}[\text{op}/x])$

op's client



Invariants of the wait-free operation must be preserved,
i.e. the operation should be “robustly safe”

E.g. safe termination of **let** `incr x = FAA(c, 1)`
relies on $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ being preserved by its client

Robust safety of verified **op**, more formally:

 $\vdash \forall \text{prog} : \text{Expr}. \text{LawyerSpec}(\text{op}) \multimap \text{PresInv}(\text{prog}[\text{op}/x])$

op's client

How to prove it for any prog?

In fact, most of clients preserve operation's invariants

What preserves the invariant $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ of `incr`?

In fact, most of clients preserve operation's invariants

What preserves the invariant $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ of `incr`?

In fact, most of clients preserve operation's invariants

What preserves the invariant $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ of `incr`?

In fact, most of clients preserve operation's invariants

`incr ()`

What preserves the invariant $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ of `incr`?

Calling the verified operation



In fact, most of clients preserve operation's invariants

```
|| let n = 5 + 5 in ||  
incr () || incr n; || incr ()
```

What preserves the invariant $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ of `incr`?

Calling the verified operation (+ in multiple threads, with any argument) ✓

In fact, most of clients preserve operation's invariants

```
inc () || let n = 5 + 5 in || while true do  
      || incr n;           || incr ()  
      ||                   || done
```

What preserves the invariant $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ of `incr`?

Calling the verified operation (+ in multiple threads, with any argument) ✓

Running infinitely ✓

In fact, most of clients preserve operation's invariants

```
let l = ref 0 in || || let n = 5 + 5 in || while true do  
l := true; || incr () || incr n; || incr ()  
( ) ( ) || || done
```

What preserves the invariant $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ of `incr`?

- Calling the verified operation (+ in multiple threads, with any argument) ✓
- Running infinitely ✓
- Getting stuck outside of the operation ✓
- Writing to *fresh* locations ✓

In fact, most of clients preserve operation's invariants

```
let l = ref 0 in || || let n = 5 + 5 in || while true do  
l := true; || incr () || incr n; || incr ()  
( ) ( ) || [n] := true || done
```

What preserves the invariant $\boxed{\exists n : \mathbb{Z}. c \mapsto n}$ of `incr`?

- Calling the verified operation (+ in multiple threads, with any argument) ✓
- Running infinitely ✓
- Getting stuck outside of the operation ✓
- Writing to *fresh* locations ✓
- Writing to *arbitrary* locations ✗

Prove robust safety with logical relations

Prove robust safety with logical relations

$RS_E(\mathbf{prog} : Expr) \approx$

execution of **prog** preserves invariants,
might get stuck,
and the returned value (if any) is “safe”

$RS_V(v : Val) \approx$

v can be used in safe programs
(*e.g.* location allocated by the client)

Prove robust safety with logical relations

$RS_E(\mathbf{prog} : Expr) \approx$

execution of **prog** preserves invariants,
might get stuck,
and the returned value (if any) is “safe”

$RS_V(v : Val) \approx$

v can be used in safe programs
(*e.g.* location allocated by the client)

Define RS_E and RS_V with more primitive Lawyer propositions to show

$$\vdash \forall \mathbf{prog}, \vec{v}. \text{NoPtrArith}(\mathbf{prog}) \multimap RS_V(\vec{v}) \multimap RS_E(\mathbf{prog}[\vec{v} / \overline{\text{freeVars}(\mathbf{prog})}])$$


Prove robust safety with logical relations

$RS_E(\mathbf{prog} : Expr) \approx$

execution of **prog** preserves invariants,
might get stuck,
and the returned value (if any) is “safe”

$RS_V(v : Val) \approx$

v can be used in safe programs
(e.g. location allocated by the client)

Define RS_E and RS_V with more primitive Lawyer propositions to show

$$\vdash \forall \mathbf{prog}, \vec{v}. \text{NoPtrArith}(\mathbf{prog}) \multimap RS_V(\vec{v}) \multimap RS_E(\mathbf{prog}[\vec{v}/\overline{\text{freeVars}(\mathbf{prog})}])$$

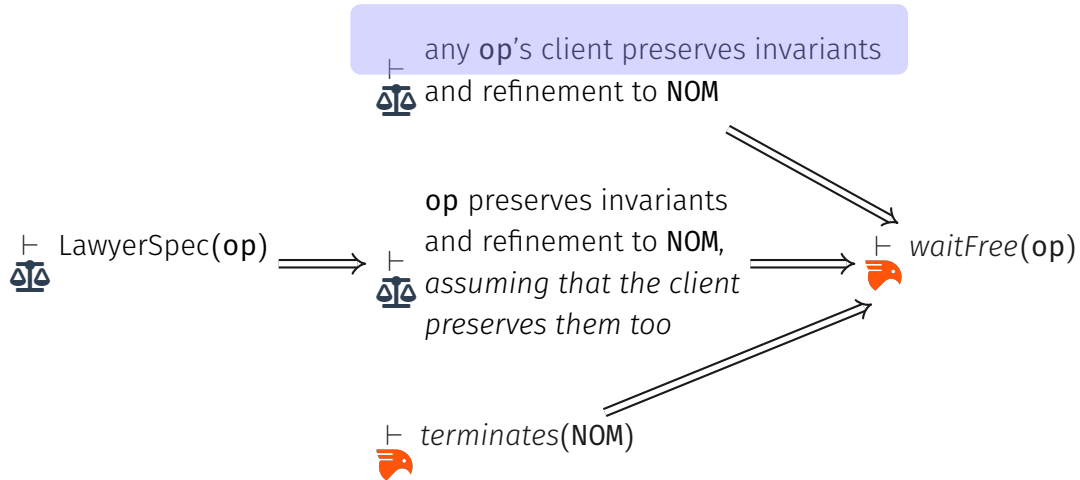

and conclude robust safety of verified **op**:

$$\vdash \forall \mathbf{prog}. \text{NoPtrArith}(\mathbf{prog}) \multimap RS_V(\mathbf{op}) \multimap RS_E(\mathbf{prog}[\mathbf{op}/x])$$

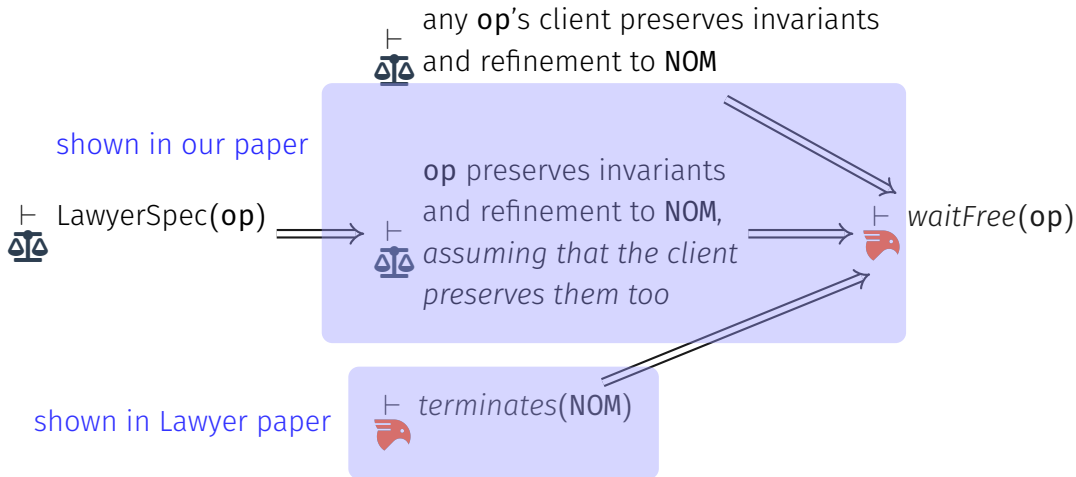

by adapting the proof of $\text{LawyerSpec}(\mathbf{op})$



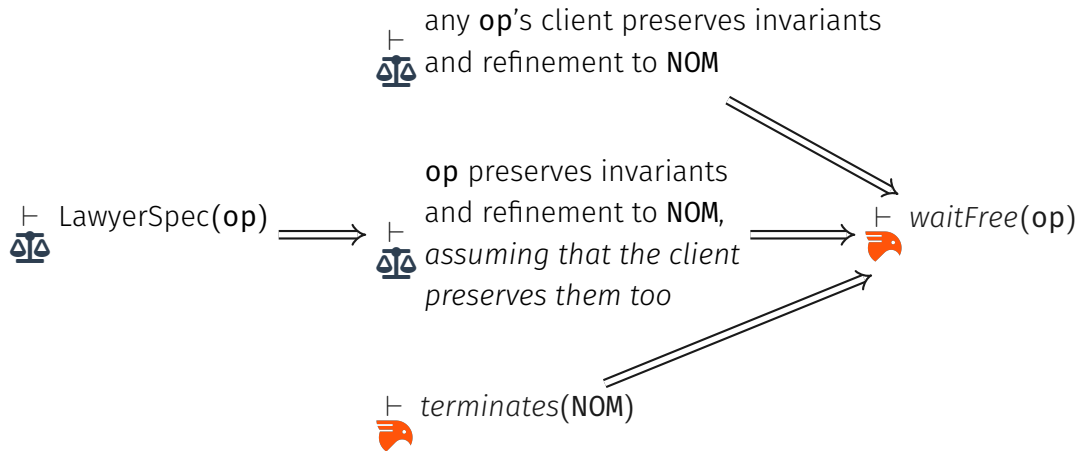
User proves spec and applies our new adequacy theorem



User proves spec and applies our new adequacy theorem



User proves spec and applies our new adequacy theorem



More complicated algorithms are not textbook-wait-free

More complicated algorithms are not textbook-wait-free

- Wait-free operation might **get stuck** if it actually inspects the input argument
e.g., `list_map f l` with wait-free `f` and `l` **not** being a list

More complicated algorithms are not textbook-wait-free

- Wait-free operation might **get stuck** if it actually inspects the input argument
e.g., `list_map f l` with wait-free `f` and `l` **not** being a list

$$\vdash \forall f. \text{WaitFreeSpec}(f) \Rightarrow \text{WaitFreeSpec}^{\text{⚡}}(\text{list_map } f)$$


More complicated algorithms are not textbook-wait-free

- Wait-free operation might **get stuck** if it actually inspects the input argument
e.g., `list_map f l` with wait-free `f` and `l` **not** being a list

$$\vdash \forall f. \text{WaitFreeSpec}(f) \Rightarrow \text{WaitFreeSpec}^{\text{⚡}}(\text{list_map } f)$$


- Realistic wait-free data structures **restrict concurrency**

```
void enq(int value) {  
    long phase = maxPhase() + 1;  
    state.set(TID, new  
        OpDesc(phase, true, true, new Node(value, TID)));  
    help(phase);  
    help_finish_enq();  
}
```

```
AtomicReferenceArray<OpDesc> state;
```

```
WFQueue () {  
    state = new AtomicReferenceArray<OpDesc>(NUM_THRDS);  
    ...  
}
```

More complicated algorithms are not textbook-wait-free

- Wait-free operation might **get stuck** if it actually inspects the input argument
e.g., `list_map f l` with wait-free `f` and `l` **not** being a list

$$\vdash \forall f. \text{WaitFreeSpec}(f) \Rightarrow \text{WaitFreeSpec}^{\text{⚡}}(\text{list_map } f)$$


- Realistic wait-free data structures **restrict concurrency**

```
void enq(int value) {  
    long phase = maxPhase() + 1;  
    state.set(TID, new  
        OpDesc(phase, true, true, new Node(value, TID)));  
    help(phase);  
    help_finish_enq();  
}
```

```
AtomicReferenceArray<OpDesc> state;  
  
WFQueue () {  
    state = new AtomicReferenceArray<OpDesc>(NUM_THRDS);  
    ...  
}
```

e.g., for 1-enqueuer, 1-dequeuer queue by Jayanti and Petrovic (2005):

$$\vdash \text{WaitFreeRestrSpec}([\text{enqueuer}, \text{dequeuer}])$$


- Proving linearizability in the same logic
(requires prophecy variables in underlying frameworks)
- Verification of universal constructions for wait-freedom
(i.e., proving $\text{waitFree}(\mathbf{UC}(\mathbf{op}))$ for fixed $\mathbf{UC}$ and any sequential $\mathbf{op}$)
- Being more flexible with amount of fuel
(needed for verifying wait-free lists)

Wait-freedom is hard to even define, harder to prove.
Solution: verify the operation with Hoare-style logic instead

$$\text{⊢} \text{WaitFreeSpec}(\text{op}) \Rightarrow \text{⊢} \text{waitFree}(\text{op})$$

Our contributions, all mechanized in Rocq:

- Definition $\text{waitFree}(\text{op})$ of op 's wait-freedom in higher-order setting;
- Specification $\text{WaitFreeSpec}(\text{op})$ in Lawyer logic and the corresponding adequacy theorem;
- Case studies for simple algorithms;
- Adapting the approach for verifying realistic, *not exactly wait-free* algorithms

Wait-freedom is hard to even define, harder to prove.
Solution: verify the operation with Hoare-style logic instead

$$\text{⊢} \text{WaitFreeSpec}(\text{op}) \Rightarrow \text{⊢} \text{waitFree}(\text{op})$$

Our contributions, all mechanized in Rocq:

- Definition $\text{waitFree}(\text{op})$ of op 's wait-freedom in higher-order setting;
- Specification $\text{WaitFreeSpec}(\text{op})$ in Lawyer logic and the corresponding adequacy theorem;
- Case studies for simple algorithms;
- Adapting the approach for verifying realistic, *not exactly wait-free* algorithms

Thank you!

Backup slides

Definition of wait-freedom

$\text{waitFree}_{C:\wp(\text{Config})}(\text{op} : \text{Val}) \triangleq \forall \text{etr} : \text{Trace}(\lambda_{\text{ref}}^{\text{conc}}).$

$$\left(\bigwedge_{_ \mapsto e \in \text{etr}[0].1} \text{validClientThread}(e, \text{op}) \right) \Rightarrow \text{etr}[0] \in C \Rightarrow \forall \tau. \text{alwaysReturnsIn}(\text{op}, \text{etr}, \tau)$$

where

- $\text{validClientThread}(e, \text{op}) \triangleq \exists e'. e = e'[\text{op}/x] \wedge \text{noLocs}(e')$

- $\text{alwaysReturnsIn}(\text{op}, \text{etr}, \tau) \triangleq \forall (E : \text{Ctx}), i.$

$$\text{Call}(\text{etr}[i].1, \tau, E, \text{op}) \Rightarrow \text{schedUntilRet}(\text{etr}, \tau, E, i) \Rightarrow \exists j \geq i. \text{Ret}(\text{etr}[j].1, \tau, E)$$

- $\text{schedUntilRet}(\text{etr}, \tau, E, i) \triangleq \forall j \geq i. j \in \text{dom } \text{etr} \Rightarrow$

$$\neg(\exists k \in [i..j]. \text{Ret}(\text{etr}[k].1, \tau, E)) \Rightarrow \exists p \geq j. \text{etr}\langle p \rangle = \tau$$

- $\text{Call}(\mathcal{E}, \tau, E, \text{op}) \triangleq \exists a \in \text{Val}. \mathcal{E}[\tau] = \text{fill } E(\text{op } a)$
- $\text{Ret}(\mathcal{E}, \tau, E) \triangleq \exists r \in \text{Val}. \mathcal{E}[\tau] = \text{fill } E r$

Wait-freedom specification in Lawyer logic

$\text{WaitFreeSpec}_C(\mathbf{op}) \triangleq \forall c : \text{Config} \in C. \text{heap_repr}(c) \Rightarrow \text{NoInfExec}(\mathbf{op}) \wedge \text{PresInv}(\mathbf{op})$,
where

- $\text{heap_repr}(c) \triangleq \bigstar_{\{(\ell, v) \mid c.2[\ell]=v\}} \ell \mapsto v$
- $\text{NoInfExec}(\mathbf{op}) \triangleq \{\mathbf{F}: \mathbb{N} \mid \forall \tau: \text{Thread}, \pi: \text{Phase}, a: \text{Val}. \{\mathbf{F} \cdot \text{bar}_\circ \pi * \text{ph}_\circ \tau \pi\} \mathbf{op} a \{\text{ph}_\circ \tau \pi\}^\tau\}$
- $\text{PresInv}(\mathbf{op}) \triangleq \forall \tau : \text{Thread}, a : \text{Val}. \{\text{RS}_V(a)\}^{\not\leq} \mathbf{op} a \{v. \text{RS}_V(v)\}^\tau$

Logical relation

$$RS_E(e) \triangleq \forall \tau. \{True\}^{\not\vdash} e \{v. RS_V(v)\}^\tau$$

$$RS_V(()) \triangleq RS_V(b : \mathbb{B}) \triangleq \dots \triangleq True$$

...

$$RS_V((v_1, v_2)) \triangleq RS_V(v_1) * RS_V(v_2)$$

$$RS_V(\ell : Loc) \triangleq \boxed{(\exists v. \ell \mapsto v * RS_V(v)) \vee freed(\ell)}$$

$$RS_V(\lambda x. e) \triangleq \forall \tau, v. \{RS_V(v)\}^{\not\vdash} (\lambda x. e)v \{r. RS_V(r)\}^\tau$$

The $\not\vdash$ triples are different from those used in Lawyer proofs:

- they don't enforce model refinement, *i.e.* only consider safety
- they allow to get stuck

Bibliography

-  Herlihy, Maurice and Nir Shavit (2012). *The Art of Multiprocessor Programming, Revised Reprint*.
-  Jayanti, Prasad and Srdjan Petrovic (2005). “Logarithmic-time single deleter, multiple inserter wait-free queues and stacks”. In: DOI: 10.1007/11590156_33.
-  Kogan, Alex and Erez Petrank (2011). “Wait-free queues with multiple enqueueers and dequeuers”. In: DOI: 10.1145/1941553.1941585.
-  Namakonov, Egor, Lars Birkedal, and Amin Timany (June 2026). “Verifying wait-freedom for concurrent higher-order programs”. In: DOI: doi.org/10.4230/LIPIcs.EC00P.2026.30.
-  Namakonov, Egor, Justus Fasse, et al. (Apr. 2026). “Lawyer: Modular Obligations-Based Liveness Reasoning in Higher-Order Impredicative Concurrent Separation Logic”. In: DOI: 10.1145/3798240.
-  Vindum, Simon Friis and Lars Birkedal (2021). “Contextual refinement of the Michael-Scott queue (proof pearl)”. In: DOI: 10.1145/3437992.3439930.